

QB 01: C Programming & Problem Solving — Question Bank with Answer Keys

Course: Programming & Problem Solving using C

Target: VTU MCA Semester 1 (IPCC / Theory + Lab)

Scheme: VTU 2022 / 2024 Scheme

Contents: Module-Wise Questions, Code Implementations, VTU Model Questions, 20 MCQs with Answer Key

Syllabus Outline (VTU 5 Modules)

- **Module 1:** Introduction to C, Primitive Data Types, Operators, Expressions, Conditional & Iterative Statements.
- **Module 2:** Arrays (1D & 2D), Matrix operations, Strings, String Manipulation Functions (`string.h`).
- **Module 3:** Modular Programming, User-Defined Functions, Parameter Passing (Pass by Value vs Reference), Recursion, Storage Classes.
- **Module 4:** Pointers, Pointer Arithmetic, Pointers with Arrays/Strings, Dynamic Memory Allocation (`malloc` , `calloc` , `realloc` , `free`), Memory Leaks & Dangling Pointers.
- **Module 5:** User-Defined Data Types (`struct` , `union` , `typedef` , `enum`), Nested Structures, File Handling (`fopen` , `fclose` , `fread` , `fwrite` , `fseek` , `ftell`), Preprocessor Directives.

SECTION 1: High-Yield MCQs with Answer Key

Q1. What is the output of `printf("%d", sizeof('A'));` in standard C (GCC/Clang)?

- A) 1
- B) 2
- C) 4 (or size of `int`)
- D) Compilation Error

Answer: C

Explanation: In C, character literals like `'A'` are treated as integer constants of type `int` , hence `sizeof('A')` equals `sizeof(int)` , which is typically 4 bytes on 32-bit/64-bit systems. (Note: in C++, it is 1 byte `char`).

Q2. Which storage class retains variable values between successive function calls?

- A) `auto`

- B) `register`
- C) `static`
- D) `extern`

Answer: C

Explanation: A `static` local variable is initialized only once and preserves its value between multiple function invocations throughout program lifetime.

Q3. What does `malloc()` return if the operating system fails to allocate the requested memory?

- A) `0`
- B) `NULL` pointer
- C) `-1`
- D) Segmentation fault

Answer: B

Explanation: If heap memory is exhausted, `malloc()` returns `NULL` (a void pointer pointing to address 0).

Q4. Which function is used to reposition the file position pointer to a specific offset in C?

- A) `rewind()`
- B) `ftell()`
- C) `fseek()`
- D) `fputs()`

Answer: C

Explanation: `fseek(FILE *stream, long int offset, int whence)` sets the file position indicator. `ftell()` returns current offset; `rewind()` sets it to start.

Q5. What is the primary difference between a `structure` and a `union` in C?

- A) Unions cannot contain pointers
- B) Structures allocate memory for all members simultaneously; Unions share a single memory space equal to the largest member
- C) Unions cannot be nested
- D) Structures cannot hold arrays

Answer: B

Explanation: All members of a structure have unique memory offsets. In a union, all members share the same starting address; the total size is the size of the largest member.

Q6. If `int arr[] = {10, 20, 30, 40}; int *ptr = arr;`, what does `*(ptr + 2)` evaluate to?

- A) 10

- B) 20
- C) 30
- D) Address of arr[2]

Answer: C

Explanation: `ptr` points to `arr[0]`. `ptr + 2` evaluates to the address of `arr[2]`. The dereference operator `*` extracts the value at that address, which is 30.

Q7. What happens if a dynamically allocated pointer is freed twice (`free(ptr); free(ptr);`)?

- A) Nothing happens
- B) Memory is doubled
- C) Undefined behavior / Double-free heap corruption error
- D) Returns NULL

Answer: C

Explanation: Freeing an already freed pointer leads to heap metadata corruption and crash vulnerabilities (CVE double-free). Always set `ptr = NULL;` after `free(ptr)`.

Q8. Which preprocessor operator is called the "stringizing" operator?

- A) `##`
- B) `#`
- C) `$`
- D) `&&`

Answer: B

Explanation: The `#` preprocessor operator converts a macro argument into a quoted string literal. The `##` operator is token pasting.

Q9. What is a "dangling pointer" in C?

- A) A pointer initialized to `NULL`
- B) A pointer pointing to a deallocated/freed memory location
- C) A pointer that points to another pointer
- D) An uninitialized pointer

Answer: B

Explanation: A dangling pointer holds the address of a memory block that has already been deallocated using `free()` or an out-of-scope stack variable.

Q10. What is the return type of `fopen()` when an error occurs (e.g., file not found for reading)?

- A) -1

- B) 0
- C) `NULL`
- D) `EOF`

Answer: C

Explanation: Standard library function `fopen()` returns a `FILE*` on success and `NULL` on failure.

SECTION 2: Short Answer Concept Questions (3–5 Marks)

Q11. Explain the difference between `calloc()` and `malloc()` with syntax and examples.

Answer: | Feature | `malloc()` | `calloc()` | |---|---|---| | **Full Name** | Memory Allocation | Contiguous Allocation | | **Arguments** | Takes 1 argument: total bytes to allocate | Takes 2 arguments: number of elements and size of each element | | **Initialization** | Does NOT initialize memory (contains garbage values) | Initializes all allocated bytes to **zero** | | **Syntax** | `ptr = (cast_type*) malloc(size);` | `ptr = (cast_type*) calloc(n, size);` | | **Performance** | Slightly faster as initialization is skipped | Slightly slower due to clearing memory to zero |

```
// Example: Allocating an array of 5 integers
int *m_ptr = (int *) malloc(5 * sizeof(int)); // Uninitialized garbage values
int *c_ptr = (int *) calloc(5, sizeof(int));  // Initialized to all 0s

free(m_ptr);
free(c_ptr);
```

Q12. Differentiate between Pass by Value and Pass by Reference in C.

Answer: In C, all function arguments are technically passed by value. However, pass by reference is simulated by passing memory addresses (pointers).

- **Pass by Value:** A copy of the actual variable is passed to the formal parameter. Changes inside the function do **not** affect the original variable.
- **Pass by Reference (Pointers):** The memory address of the variable is passed. Changes made through dereferencing affect the original variable in the caller.

```
#include <stdio.h>

// Pass by Value
void modifyVal(int x) {
    x = 100; // Original variable unchanged
}

// Pass by Reference (via pointer)
void modifyRef(int *x) {
    *x = 100; // Original variable changed!
}

int main() {
    int a = 10;
    modifyVal(a);
    printf("After modifyVal: %d\n", a); // Prints 10

    modifyRef(&a);
    printf("After modifyRef: %d\n", a); // Prints 100
    return 0;
}
```

Q13. What is recursion? What are its prerequisites? Write a recursive function to compute the factorial of a number.

Answer: Recursion is a programming technique where a function calls itself directly or indirectly to solve a smaller instance of the same problem.

Prerequisites for a valid recursive function:

1. **Base Case (Termination condition):** A condition under which the function stops calling itself, preventing infinite recursion and stack overflow.
2. **Recursive Step:** Logic that reduces the problem towards the base case.

```
#include <stdio.h>

long long factorial(int n) {
    // Base Case
    if (n ≤ 1) {
        return 1;
    }
    // Recursive Step
    return n * factorial(n - 1);
}

int main() {
    int num = 5;
    printf("Factorial of %d = %lld\n", num, factorial(num)); // Output: 120
    return 0;
}
```

Time Complexity: $O(N)$

Auxiliary Space (Call Stack): $O(N)$

SECTION 3: Long Answer / VTU Model Exam Questions (8–12 Marks)

Q14. [VTU Model QP - Module 4]

(a) What are Pointers? Explain Pointer Arithmetic with suitable illustrations.

(b) Write a complete C program to swap two arrays of size N using pointers without temporary arrays.

Answer:

Part (a): Pointers & Pointer Arithmetic

A **pointer** is a variable that stores the memory address of another variable.

Syntax: `data_type *pointer_name;`

Valid Pointer Arithmetic Operations:

1. **Incrementing (`ptr++`)**: Moves the pointer to the next memory location of its base type.

$$\text{New Address} = \text{Current Address} + (1 \times \text{sizeof(datatype)})$$
2. **Decrementing (`ptr--`)**: Moves to previous element location.
3. **Addition of Integer (`ptr + n`)**: Skips n elements forward.
4. **Subtraction of Integer (`ptr - n`)**: Moves n elements backward.
5. **Subtraction of two pointers of the same type (`ptr2 - ptr1`)**: Returns the number of elements between them. (Note: Adding two pointers, multiplying, or dividing pointers is illegal in C).

Part (b): C Program to Swap Two Arrays Using Pointers

```

#include <stdio.h>

void swapArrays(int *arr1, int *arr2, int n) {
    for (int i = 0; i < n; i++) {
        // Swap values using pointer dereferencing and arithmetic
        int temp = *(arr1 + i);
        *(arr1 + i) = *(arr2 + i);
        *(arr2 + i) = temp;
    }
}

void printArray(const char *name, int *arr, int n) {
    printf("%s: ", name);
    for (int i = 0; i < n; i++) {
        printf("%d ", *(arr + i));
    }
    printf("\n");
}

int main() {
    int n = 5;
    int a[5] = {1, 2, 3, 4, 5};
    int b[5] = {10, 20, 30, 40, 50};

    printf("--- Before Swapping ---\n");
    printArray("Array A", a, n);
    printArray("Array B", b, n);

    swapArrays(a, b, n);

    printf("\n--- After Swapping ---\n");
    printArray("Array A", a, n);
    printArray("Array B", b, n);

    return 0;
}

```

Q15. [VTU Model QP - Module 5]

(a) Explain file operations in C: Modes of opening, sequential vs random access.

(b) Write a C program to copy contents of one text file into another, converting all lowercase characters to uppercase.

Answer:

Part (a): File Operations & Modes

In C, files are accessed through a `FILE` structure pointer managed by the C standard I/O library (`<stdio.h>`).

Common File Modes:

- `"r"`: Open existing file for reading. Returns `NULL` if not found.

- `"w"` : Create or overwrite file for writing.
- `"a"` : Open for appending at the end of the file.
- `"r+"` : Open for reading and writing (file must exist).
- `"w+"` : Create/truncate for reading and writing.
- `"rb"` , `"wb"` , `"ab"` : Binary file equivalents.

Access Types:

- **Sequential Access:** Reads or writes byte-by-byte in chronological order using `fgetc()` , `fgets()` , `fprintf()` .
- **Random Access:** Directly positions to any byte offset using:
 - `fseek(fp, offset, SEEK_SET / SEEK_CUR / SEEK_END)`
 - `ftell(fp)` : Returns current position offset in bytes.
 - `rewind(fp)` : Moves file pointer back to start (`offset = 0`).

Part (b): C Program — File Copy with Lowercase to Uppercase Conversion


```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

int main() {
    FILE *srcFile, *destFile;
    char ch;
    const char *sourcePath = "source.txt";
    const char *destPath = "destination.txt";

    // 1. Create a sample source file for demonstration
    srcFile = fopen(sourcePath, "w");
    if (srcFile == NULL) {
        printf("Error creating source file.\n");
        return 1;
    }
    fputs("Hello VTU MCA Students! Welcome to C Programming File I/O.\n", srcFile);
    fclose(srcFile);

    // 2. Open source file in read mode and destination in write mode
    srcFile = fopen(sourcePath, "r");
    if (srcFile == NULL) {
        perror("Error opening source file");
        return 1;
    }

    destFile = fopen(destPath, "w");
    if (destFile == NULL) {
        perror("Error opening destination file");
        fclose(srcFile);
        return 1;
    }

    // 3. Read character by character, transform case, and write
    while ((ch = fgetc(srcFile)) != EOF) {
        fputc(toupper((unsigned char)ch), destFile);
    }

    printf("File copied and transformed to uppercase successfully!\n");

    // 4. Always close all file handles
    fclose(srcFile);
    fclose(destFile);

    return 0;
}
```

Q16. [VTU Model QP - Module 5]

Define a structure `Student` with fields `usn`, `name`, `marks[3]`, and `total`. Write a program to read records for N students, calculate total marks, and display the details of the student who scored the

highest total.

Answer:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_STUDENTS 100

typedef struct {
    char usn[15];
    char name[50];
    float marks[3];
    float total;
} Student;

void calculateTotal(Student *s) {
    s->total = s->marks[0] + s->marks[1] + s->marks[2];
}

int main() {
    int n;
    Student students[MAX_STUDENTS];

    printf("Enter number of students (1 to %d): ", MAX_STUDENTS);
    if (scanf("%d", &n) != 1 || n <= 0 || n > MAX_STUDENTS) {
        printf("Invalid student count.\n");
        return 1;
    }

    // Read details
    for (int i = 0; i < n; i++) {
        printf("\n--- Entering details for Student %d ---\n", i + 1);
        printf("Enter USN: ");
        scanf("%14s", students[i].usn);
        printf("Enter Name: ");
        scanf(" %49[^\n]", students[i].name);

        printf("Enter marks for 3 subjects: ");
        scanf("%f %f %f", &students[i].marks[0], &students[i].marks[1], &students[i].marks[2]);

        calculateTotal(&students[i]);
    }

    // Find highest scorer
    int topperIndex = 0;
    for (int i = 1; i < n; i++) {
        if (students[i].total > students[topperIndex].total) {
            topperIndex = i;
        }
    }

    // Display topper details
    printf("\n===== \n");
    printf("          VTU MCA TOPPER DETAILS          \n");
    printf("===== \n");

```

```
printf("USN      : %s\n", students[topperIndex].usn);
printf("Name      : %s\n", students[topperIndex].name);
printf("Marks (3 Sub): %.2f, %.2f, %.2f\n",
       students[topperIndex].marks[0],
       students[topperIndex].marks[1],
       students[topperIndex].marks[2]);
printf("Total Marks : %.2f / 300\n", students[topperIndex].total);
printf("Percentage  : %.2f%%\n", (students[topperIndex].total / 3.0));
printf("=====\n");

return 0;
}
```

Key Takeaway Checklist for Exam Day

- [] Review pointer syntax and dereference operator precedence (`*p++` vs `(*p)++`).
- [] Memorize format specifiers (`%zu` for `size_t` , `%p` for memory address, `%[^\n]` for strings with spaces).
- [] Remember to check for `NULL` after every `malloc()` / `fopen()` call.
- [] Always call `free()` on dynamically allocated memory and `fclose()` on open file pointers.